

Object Oriented Programming in C++

Virtual Functions, Type Casting, and Exception Handling

UTSA ROY

Department Of Computer Science And Engineering
Indian Institute Of Technology, Kharagpur

Virtual Functions

Runtime Polymorphism

- Polymorphism allows one interface to represent multiple behaviors.
- Two types of polymorphism in C++:
 - Compile-time polymorphism
 - Function overloading
 - Operator overloading
 - Runtime polymorphism
 - Implemented using virtual functions
- In runtime polymorphism, the function call is resolved during program execution.

Function Overriding

- Function overriding occurs when a derived class redefines a function of the base class.
- Conditions for overriding:
 - Same function name
 - Same parameter list
 - Function must be accessible

Example:

```
1 class Base {  
2 public:  
3     void show() {  
4         cout << "Base class function";  
5     }  
6 };  
7  
8 class Derived : public Base {  
9 public:  
10     void show() {  
11         cout << "Derived class function";  
12     }  
13 };
```

Static Binding Problem

```
1 Base* ptr;  
2 Derived obj;  
3 ptr = &obj;  
4 ptr->show();
```

Output:

```
1 Base class function
```

- The compiler decides the function based on the type of pointer.
- The pointer type here is Base*.
- This behavior is called **Static Binding (Early Binding)**.

Virtual Functions

- A virtual function is a member function of a class declared using the keyword `virtual`.
- It tells the compiler that the function call should be resolved at runtime.

Syntax:

```
1 class Base {  
2 public:  
3     virtual void show() {  
4         cout << "Base class function" << endl;  
5     }  
6 };
```

- When a function is declared virtual in a base class, it allows derived classes to override the function dynamically.
- The decision about which function to execute is made during program execution, not during compilation.
- This mechanism is called **Dynamic Binding (Late Binding)**.

Example of Virtual Function

```
1 class Base {
2 public:
3     virtual void show() {
4         cout << "Base class function";
5     }
6 };
7
8 class Derived : public Base {
9 public:
10     void show() {
11         cout << "Derived class function";
12     }
13 };
14
15 int main() {
16     Base* ptr;
17     Derived obj;
18     ptr = &obj;
19     ptr->show();
20 }
```

Output: Derived class function

- The function is selected based on the object type at runtime.

Base Class Pointer and Virtual Functions

- Virtual functions are typically used with base class pointers.
- A base class pointer can point to an object of a derived class.

Example:

```
1 Base* ptr;  
2 Derived obj;  
3 ptr = &obj;
```

- When the function is declared virtual, the function call is resolved based on the actual object type.

```
1 ptr->show();
```

- This mechanism enables runtime polymorphism.

Virtual Table

- The implementation of virtual functions is commonly based on a structure called the **Virtual Table (vtable)**.
- Each class with virtual functions maintains a table of function pointers.
- The object contains a hidden pointer called **vp_ptr** that points to the vtable.

Execution process:

- 1 Object is created
- 2 vp_ptr points to the class vtable
- 3 Function call is resolved using the vtable

Conceptual representation:

```
1 Object
2   -> vp_ptr
3   -> vtable
4       |-> Base::show()
5       |-> Derived::show()
```

- This mechanism enables dynamic binding.

Pure Virtual Functions

- Sometimes a base class provides only the interface, not the implementation.
- Such functions are declared as pure virtual functions.

Syntax:

```
1 virtual void display() = 0;
```

Example:

```
1 class Shape {  
2 public:  
3     virtual void draw() = 0;  
4 };
```

- A class containing at least one pure virtual function becomes an abstract class.

Abstract Classes

- An abstract class is a class that cannot be instantiated.
- It is designed only to act as a base class.

Example:

```
1 class Shape {  
2 public:  
3     virtual void draw() = 0;  
4 };  
5  
6 class Circle : public Shape {  
7 public:  
8     void draw() {  
9         cout << "Drawing Circle";  
10    }  
11};
```

- Objects can only be created for the derived classes.

Rules for Virtual Functions

- Virtual functions must be member functions of a class.
- They cannot be static functions.
- They are usually accessed using base class pointers.
- The `virtual` keyword is declared in the base class only.
- Derived classes may redefine the function.
- Constructors cannot be virtual, but destructors can be virtual.

Type Casting

Type Casting

- Type casting refers to converting a variable from one data type to another.
- In C++, type conversion can occur in two ways:
 - **Implicit Type Conversion**
 - Done automatically by the compiler
 - **Explicit Type Conversion (Type Casting)**
 - Performed manually by the programmer

Example:

```
1 int a = 10;  
2 double b;  
3 b = a;    // implicit conversion
```

- Here the integer value is automatically converted into a double.

Explicit Type Casting

- Explicit type casting is used when the programmer wants to force a type conversion.

Example:

```
1 int a = 5;  
2 int b = 2;  
3 double result;  
4 result = (double)a / b;
```

Output:

```
1 2.5
```

Without casting:

```
1 2
```

- Explicit casting ensures correct numerical computation.

C-Style Casting

- In older C/C++ programs, casting was done using C-style syntax.

Syntax:

```
1 (type) expression
```

Example:

```
1 float x = 10.5;  
2 int y;  
3 y = (int)x;
```

Output:

```
1 y = 10
```

- C-style casting can be unsafe because it does not clearly indicate the type of conversion.

C++ Casting Operators

- Modern C++ provides four type-safe casting operators.

- **1. static_cast**

- Used for standard type conversions.

```
1 static_cast<type>(expression)
```

Example:

```
1 double x = 10.5;  
2 int y = static_cast<int>(x);
```

- **2. dynamic_cast**

- Used for safe downcasting in inheritance.
 - It is mainly used for downcasting (base → derived)
 - Works only when the base class has at least one virtual function

Example

```
1 #include <iostream>
2 using namespace std;
3
4 class Base {
5 public:
6     virtual void show() {}
7 };
8
9 class Derived : public Base {
10 public:
11     void display() {
12         cout << "Derived class";
13     }
14 };
15
16 int main() {
17     Base* ptr = new Derived;
18
19     Derived* dptr = dynamic_cast<Derived*>(ptr);
20
21     if(dptr != nullptr)
22         dptr->display();
23     else
24         cout << "Conversion failed";
25 }
```

- Returns valid pointer if successful, otherwise `nullptr`

Other Casting Operators

3. `const_cast`

- Used to add or remove `const` qualifier.

Example:

```
1 const int x = 10;  
2 int* p = const_cast<int*>(&x);
```

4. `reinterpret_cast`

- Performs low-level reinterpretation of bits.

Example:

```
1 int a = 65;  
2 char* p = reinterpret_cast<char*>(&a);
```

- These casting operators provide better type safety than C-style casting.

Exception Handling

Exception Handling

- During program execution, runtime errors may occur.
- Examples:
 - Division by zero
 - Invalid memory access
 - File opening failure
 - Invalid input
- Such errors may cause the program to terminate abnormally.
- C++ provides a mechanism called **Exception Handling** to detect and handle these errors.
- Exception handling improves program reliability and maintainability.

Need for Exception Handling

- Traditional error handling techniques have several limitations:
 - Error checking code gets mixed with normal program logic
 - Makes programs harder to read and maintain
 - Difficult to propagate errors across functions

Example (traditional approach):

```
1 if(b == 0)
2 {
3     cout << "Error: Division by zero";
4 }
```

- Exception handling provides a structured mechanism to manage runtime errors.

Basic Exception Handling Mechanism

- C++ exception handling uses three keywords:

try throw catch

General syntax:

```
1 try
2 {
3     // code that may generate an exception
4 }
5 catch(exception_type e)
6 {
7     // exception handling code
8 }
```

- The try block contains risky code.
- The throw statement signals an exception.
- The catch block handles the exception.

The throw Statement

- The `throw` keyword is used to signal that an exception has occurred.

Syntax:

```
1 throw expression;
```

Example:

```
1 int b = 0;  
2 if (b == 0)  
3 {  
4     throw "Division by zero error";  
5 }
```

- When `throw` is executed, program control immediately transfers to the corresponding `catch` block.

Basic Example of Exception Handling

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int a = 10;
7     int b = 0;
8
9     try
10    {
11        if(b == 0)
12            throw "Division by zero error";
13        cout << a / b;
14    }
15    catch(const char* msg)
16    {
17        cout << msg;
18    }
19
20    return 0;
21 }
```

Output: Division by zero error

Multiple Catch Blocks

- A program can have multiple catch blocks to handle different exceptions.

Example:

```
1 try
2 {
3     throw 10;
4 }
5 catch(int e)
6 {
7     cout << "Integer exception";
8 }
9 catch(double e)
10 {
11     cout << "Double exception";
12 }
```

- The catch block whose type matches the thrown exception will execute.

Catch-All Handler

- Sometimes we may not know the exact type of exception.
- C++ provides a catch-all handler.

Syntax:

```
1 catch(...)  
2 {  
3     // handles any exception  
4 }
```

Example:

```
1 try  
2 {  
3     throw 3.14;  
4 }  
5 catch(...)  
6 {  
7     cout << "Unknown exception caught";  
8 }
```

Exception Propagation

- An exception may occur inside a called function.
- If it is not handled there, it is propagated to the calling function.

Example:

```
1 void func()  
2 {  
3     throw 10;  
4 }  
5  
6 int main()  
7 {  
8     try  
9     {  
10        func();  
11    }  
12    catch(int e)  
13    {  
14        cout << "Exception caught";  
15    }  
16 }
```

- This process is known as exception propagation.

Standard Exceptions

- C++ provides several built-in exception classes.
- They are defined in the `<stdexcept>` library.
- Common examples:
 - `runtime_error`
 - `out_of_range`
 - `invalid_argument`
 - `bad_alloc`

Example:

```
1 #include <stdexcept>
2 throw runtime_error("Runtime error occurred");
```

Advantages of Exception Handling

- Separates error handling code from normal program logic
- Improves program readability
- Enables handling different types of runtime errors
- Supports exception propagation across functions
- Helps develop robust and reliable software

Thank you!